

Muon 实践：什么时候用，怎么用，别怎么用

第六讲：参数分组、缩放、系统成本与变体地图

Muon 的核心公式不长，但真正使用时要处理很多工程细节：哪些参数交给 Muon，哪些仍用 AdamW；如何做 weight decay；正交化步数、缩放、QKV 和分布式开销该怎样理解。

本讲要回答

- Muon 实践中的参数分组怎么做？
- weight decay 和 update scaling 如何理解？
- Muon 的主要代价和边界在哪里？
- 后续变体大致在改哪一块？

本讲先不做

- 不声称 Muon 在所有任务都优于 AdamW。
- 不替代具体项目里的超参数搜索。
- 不深入每篇 2026 变体的证明。

这组讲义会固定符号： F 表示当前训练目标，训练时通常 $F = \hat{L}_N$ ； g_t 是 mini-batch 梯度， m_t 是一阶动量。

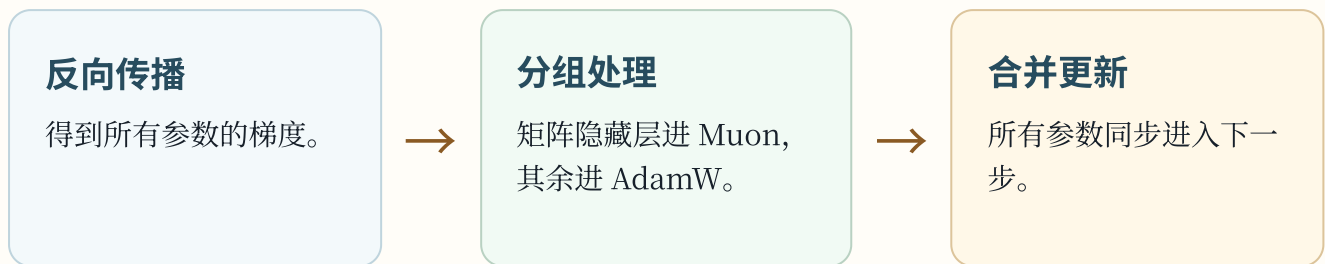
实践里的 Muon 通常是混合优化器

一个常见分组是：

参数类型	常见处理	理由
隐藏层二维矩阵	Muon	可以利用矩阵结构
bias、norm、标量、向量	AdamW 或不做 decay	不适合矩阵正交化
embedding、输出头	AdamW	经验上常与隐藏层动态不同
卷积核	可展平成二维后尝试 Muon	把空间和通道维合并成矩阵

最重要的实践观念：Muon 是给一部分矩阵参数的更新规则，不是全模型万能替换。

每一步里 AdamW 组和 Muon 组并行更新



Muon 组内部通常是：

$$G_t \rightarrow M_t \rightarrow \text{Newton-Schulz} \rightarrow U_t \rightarrow W_{t+1}$$

AdamW 组内部仍然是：

$$g_t \rightarrow \hat{m}_t, \hat{v}_t \rightarrow \theta_{t+1}$$

这两条支路不是互相排斥，而是各管一类参数。

Muon 里也要把 weight decay 当成独立收缩

和 AdamW 一样，实践中更清楚的写法是先做 decoupled weight decay：

$$W_t^{\text{decay}} = (1 - \eta\lambda)W_t$$

然后对梯度方向做 Muon 更新：

$$W_{t+1} = W_t^{\text{decay}} - \eta \cdot \text{scale}(W) \cdot U_t$$

为什么解耦

weight decay 表示对参数本身的收缩，不应该被正交化子程序混进去。

为什么有 scale

Orth 后的矩阵范数与形状相关，实际配方常需要层形状相关的缩放。

把 decay、learning rate、orthogonalization 分开理解，调参时会清楚很多。

Orth(M) 的范数由矩阵形状决定

如果 U 是 $m \times n$ 的半正交矩阵，那么它的谱范数约为 1，Frobenius 范数约为：

$$\|U\|_F \approx \sqrt{\min(m, n)}$$

这意味着同一个 η 在不同形状的层上可能产生不同总更新量。因此许多 Muon 配方会加上形状相关或层相关 scaling。

谱范数稳定

最大奇异值被控制在约 1。

总能量变化

Frobenius 范数随短边维度增长。

所以学习率不是唯一尺度，更新归一化后的 shape scale 也会进入配方。

Muon 的“步长”不只是 η ，还包括正交化后矩阵更新如何按层缩放。

矩阵怎么切，会影响 Muon 看到的几何

Transformer 里 Q、K、V 有时被合成一个大矩阵实现。但 Muon 的正交化会把整个矩阵作为一个几何对象。

合并 QKV

一个大矩阵一起正交化，Q/K/V 的子空间会被混在一个谱结构里。

分开 Q、K、V

三个矩阵各自形成动量并正交化，经验上常被认为更合理。

这说明 Muon 对模块边界更敏感：如何把参数组织成矩阵，会影响优化器的行为。

Adam 的逐坐标缩放对矩阵切分不那么敏感；Muon 的矩阵正交化则会关心你把哪些维度放进同一个矩阵。

Muon 的额外成本主要来自正交化矩阵乘法

Newton-Schulz 每一步需要若干矩阵乘法。若做 q 步，额外成本大致随 q 、矩阵短边维度和层数增加。

为什么可行

它使用矩阵乘法，适合 GPU；相比每步 SVD 便宜很多。

为什么仍有成本

非元素级优化器更难和分片、通信、量化状态等系统机制无缝结合。

因此大规模训练里，Muon 不只是数学问题，也是系统实现问题。

不要只问“Muon 方向好不好”，还要问“这个方向在我的训练系统里能不能便宜地算出来”。

Muon 的边界比公式更重要

场景	需要谨慎的原因
参数不是二维矩阵	Muon 的核心正交化对象不成立
embedding / 输出头	经验上常用 AdamW 更稳
小模型或小 batch	正交化开销和噪声特性可能不划算
微调 / RLHF	预训练上的优势不自动迁移
分布式分片复杂	矩阵级更新需要额外系统支持

Muon 是一个强工具，但不是“看到矩阵就上”。先保证 AdamW 基线被认真调过，再判断 Muon 的收益。

后续 Muon 变体大多在改四个位置

改动量信号

Nesterov、方差自适应、梯度差分加速等。

改正交化子程序

更少 Newton-Schulz 步数、更好多项式、低秩近似。

改谱变换

不一定把奇异值全压成 1，而是保留部分谱信息。

改系统状态

量化、分布式、张量化、LoRA 低秩结构。

读新论文时可以先问：它是在改 $G \rightarrow M$ ，还是改 $M \rightarrow U$ ，还是改 U 的缩放与部署？

这张地图比背名字更重要。名字会变，四个改动位置相对稳定。

从 SGD 到 Muon，我们完成了一条主线

1. SGD：用 mini-batch 梯度近似训练集梯度。
2. Momentum：在时间上平均梯度，降低单步噪声。
3. Adam：在坐标上估计尺度，构造对角自适应步长。
4. Shampoo/SOAP：开始把矩阵左右空间纳入优化器状态。
5. Muon：对矩阵动量做正交化，用谱更平坦的方向更新隐藏层。

SGD → Momentum → AdamW → Shampoo/SOAP → Muon

你现在应该能读懂

Muon 更新方程、Newton-Schulz 迭代、Orth(M) 的谱意义、与 AdamW 的分工。

下一阶段可以做

读 Muon 原文和后续变体论文，把每篇论文放到这张优化器地图上。

Muon 的关键不是“新 optimizer 很神奇”，而是它把矩阵结构、动量信号和 GPU 友好的正交化算法接在了一起。